

**FEDERAL STATE AUTONOMOUS EDUCATIONAL INSTITUTION FOR HIGHER
EDUCATION NATIONAL RESEARCH UNIVERSITY**

«HIGHER SCHOOL OF ECONOMICS»

Faculty of Computer Science

Yuqi Xia

Name, Surname

Алгоритм генетического видообразования для решения задач комбинаторной
оптимизации

Russian title

Genetic Speciation Algorithm for Addressing Combinatorial Optimization Problems

English title

Term paper

Field of study 01.04.02 «Applied Mathematics and Informatics»

Program: «Data Science»

Student

Yuqi Xia

Name

Supervisor

Majid Sohrabi

Name

Moscow, 2026

Content

Abstract.....	2
1 Introduction	3
2 Inspiration.....	5
3 Proposed GSA	7
3.1 Solution Representation	7
3.2 Dynamic Species Formation	8
3.3 Evolution Within Species.....	10
3.4 Inter-Species Interaction and Diversity Maintenance.....	13
3.5 Overall Framework	14
4 Experimental Results.....	17
4.1 Experimental Setup.....	17
4.2 Overall Performance	18
4.3 Convergence Behavior.....	19
4.4 Robustness and Scalability	20
4.5 Species Dynamics	21
4.6 Discussion	22
5 Conclusion and Future Work.....	23
Reference.....	25

Abstract

Premature convergence is a major limitation of genetic algorithms for combinatorial optimization, as the population may quickly collapse into a single dominant search direction. This paper proposes a Genetic Speciation Algorithm (GSA), a speciation-centered evolutionary method that preserves multiple structural search niches during optimization. Instead of evolving one fully mixed population, GSA dynamically partitions individuals into species according to solution-structure similarity and fitness information, allowing each species to explore and refine a distinct region of the search space. For capacitated vehicle routing problems, candidate solutions are represented as customer permutations, and species are formed based on route-edge similarity. Experiments on Augerat benchmark instances show that GSA consistently outperforms both a standard genetic algorithm and a non-speciated genetic engineering algorithm. The mean gaps to the best known solutions are reduced from 15.7%, 19.1%, and 24.3% to 5.1%, 10.7%, and 13.7%, respectively. These results demonstrate that speciation is an effective mechanism for maintaining diversity, improving robustness, and enhancing solution quality in evolutionary combinatorial optimization.

Keywords: genetic algorithm, speciation, population clustering, combinatorial optimization, vehicle routing problem.

Repository: <https://github.com/Uuu-7/Genetic-Speciation-Algorithm.git>

1 Introduction

Combinatorial optimization problems arise in many practical settings, including vehicle routing, scheduling, logistics planning, and resource allocation. These problems usually involve a large number of discrete decisions, and their search spaces often grow rapidly with the problem size, making exact algorithms difficult to apply within a reasonable time ^[1]. Metaheuristic methods are therefore widely used to obtain high-quality approximate solutions.

Genetic algorithms are a common choice for combinatorial optimization because they search with a population of candidate solutions and can be adapted to different discrete representations ^[2,3]. Through selection, crossover, and mutation, they can explore different regions of the search space and combine useful solution components across generations. However, a standard genetic algorithm often evolves the whole population as a single mixed group. When a small number of strong individuals dominate early, the population may quickly lose diversity and converge to a local optimum.

Recent work on the Genetic Engineering Algorithm (GEA) introduced genetic-engineering-inspired operators to improve the search ability of standard genetic algorithms ^[4]. However, GEA still evolves the population as a single mixed group, so structurally different search directions may not be sufficiently preserved. This issue is closely related to solution structure. In combinatorial optimization, two solutions with similar objective values may still represent very different structural patterns. In the capacitated vehicle routing problem, for example, the quality of a solution depends not

only on the order of customers, but also on the route edges and customer adjacency relationships. If the population becomes concentrated around one route structure too early, other promising structures may disappear before they are sufficiently explored.

To address this problem, this paper introduces speciation into the genetic search process. Speciation divides the population into groups of structurally similar individuals, where each group is treated as a species. Instead of forcing all individuals to compete in one global population, each species mainly evolves within its own search niche. This design helps preserve multiple search directions and reduces the risk that one dominant structure takes over the entire population.

Based on this idea, this paper propose a Genetic Speciation Algorithm for combinatorial optimization. The proposed method dynamically forms species according to structural similarity and fitness information. For the capacitated vehicle routing problem, each solution is represented as a customer permutation, and species are mainly formed according to route-edge similarity. In this way, the grouping process reflects meaningful routing structures rather than arbitrary distance measures.

The main purpose of GSA is to reorganize the population so that different solution structures can survive and improve during the search. This distinguishes it from approaches that only modify genetic operators while keeping a single mixed population. By maintaining several species, GSA provides a more direct mechanism for preserving structural diversity. Experimental results on standard CVRP benchmark instances show that this mechanism improves both solution quality and robustness compared with a standard genetic algorithm and a non-speciated evolutionary algorithm.

2 Inspiration

Speciation is a basic mechanism in natural evolution. A population may gradually split into different groups when individuals adapt to different environments or become partially isolated from one another. These groups then evolve along different directions. This process helps preserve diversity and prevents the whole population from being dominated by a single genetic pattern. This idea is useful for genetic algorithms. In a standard genetic algorithm, all individuals usually belong to one mixed population. Selection gives better individuals a higher chance of reproduction, which can improve convergence speed. However, it can also make the population become similar too quickly. Once most individuals share the same structure, crossover and mutation may produce only limited variation, and the search can become trapped around a local optimum.

For combinatorial optimization, this loss of diversity is especially harmful because useful information is often structural. In routing problems, for example, important information lies in route segments, customer adjacency, and depot connections. Two solutions may have similar costs but contain different route edges. These different edge structures may lead to different promising regions of the search space. If one structure dominates too early, the algorithm may lose the chance to explore other useful alternatives.

Speciation provides a direct way to reduce this risk. By dividing the population into several species, the algorithm allows different groups of individuals to preserve different structures. Individuals mainly compete and reproduce with structurally

similar individuals inside the same species. As a result, a strong solution in one species does not immediately remove other promising structures in other species.

In the proposed GSA, species are formed dynamically according to solution-structure similarity and fitness information. For the capacitated vehicle routing problem, route-edge similarity is used as the main criterion. Individuals that share many route edges are placed in the same species, while individuals with different edge structures are encouraged to remain in different species. This makes each species correspond to a meaningful search niche.

The role of speciation in GSA is therefore twofold. First, it preserves diversity by keeping several structural patterns in the population. Second, it organizes the search by allowing each species to refine its own pattern. The whole population can explore multiple promising regions, while each species performs focused improvement within its own niche.

3 Proposed GSA

This section presents the proposed Genetic Speciation Algorithm (GSA). The main idea is to reorganize the population into dynamically updated species during evolution. Each species contains individuals with similar chromosome structures and searches within a specific region of the solution space. This organization preserves multiple search directions while still allowing limited information exchange among species.

3.1 Solution Representation

This paper applies GSA to the capacitated vehicle routing problem (CVRP). A CVRP instance includes a depot, a set of customers, customer demands, and vehicles with identical capacity. Each customer must be visited exactly once, and the total demand assigned to a vehicle cannot exceed its capacity. The objective is to minimize the total travel distance.

Each individual is encoded as a permutation of all customers:

$$\pi = [c_1, c_2, \dots, c_n], \quad (1)$$

where each customer appears exactly once. The depot is not explicitly included in the chromosome. To evaluate an individual, customers are scanned from left to right. A customer is appended to the current route if the resulting demand remains within the capacity limit. Otherwise, the current route returns to the depot, and a new route starts with the current customer.

If the decoded solution contains routes $R_1, R_2, \dots, R_m$, the fitness is the total

travel distance:

$$f(\pi) = \sum_{k=1}^m L(R_k), \quad (2)$$

where $L(R_k)$ denotes the travel distance of route R_k . Since CVRP is a minimization problem, a smaller fitness value indicates a better solution. This representation allows genetic operations to be applied directly to customer sequences while capacity-feasible routes are generated during evaluation.

3.2 Dynamic Species Formation

Dynamic species formation is the core component of GSA. Instead of evolving the entire population as one mixed group, the algorithm periodically divides individuals into several species. Individuals in the same species share similar chromosome structures, while individuals in different species represent distinct search directions.

3.2.1 Chromosome-Edge Representation

For permutation-based routing problems, structural similarity is better represented by adjacency relationships than by absolute customer positions. For a chromosome $\pi = [c_1, c_2, \dots, c_n]$, an edge set is constructed from the sequence $[0, c_1, c_2, \dots, c_n, 0]$, where 0 denotes the depot. Each edge is stored as an undirected canonical pair.

Depot edges are included because customers near the depot may provide useful structural information. The edge set is constructed directly from the chromosome sequence rather than from the decoded routes. The decoded routes are used for fitness evaluation, while the chromosome-edge set is used for clustering, diversity

measurement, novelty detection, and structural information extraction.

3.2.2 Hybrid Distance

The structural difference between two individuals A and B is measured using Jaccard distance:

$$D_{edge}(A, B) = 1 - \frac{|E(A) \cap E(B)|}{|E(A) \cup E(B)|}. \quad (3)$$

A smaller value indicates that two individuals share more chromosome edges. Fitness information is also included to distinguish individuals with similar edge patterns but different solution quality. Individuals are ranked in ascending order of fitness, and the ranks are normalized to the interval $[0, 1]$ by dividing them by $\max(N - 1, 1)$. The fitness-rank distance is the absolute difference between the normalized ranks.

The final hybrid distance is:

$$D(A, B) = 0.80 * D_{edge}(A, B) + 0.20 * D_{rank}(A, B). \quad (4)$$

Thus, the hybrid-distance weight alpha is 0.20. Structural similarity remains the main criterion, while fitness ranks provide additional information about solution quality. To reduce computational overhead, edge sets are converted into a binary matrix, and pairwise intersections are calculated through matrix multiplication.

3.2.3 Wishart-Style Clustering Procedure

Species are formed using a Wishart-style density clustering procedure ^[5]. The effective number of nearest neighbors is $k = \min(\max(4, \lfloor 0.08N \rfloor), N - 1)$. For the experimental population size $N = 200$, this gives $k = 16$. Individuals are

processed in ascending order of their distance to the k -th nearest neighbor, so dense regions are considered first.

When processing an individual, a new species is created if none of its processed neighbors has been assigned. The individual joins an existing species if its assigned neighbors belong to one species. If its assigned neighbors belong to multiple species, these species are merged.

3.2.4 Species-Size Control and Reclustering

Provisional species larger than 45% of the population are recursively divided using a two-medoids procedure. Species with fewer than three individuals are absorbed into their nearest larger species. The number of species is limited to five. If more than five species remain, smaller species are merged into their nearest retained species.

Species are formed at the beginning of the search and recomputed every 20 generations. Reclustering is also forced after each diversity burst. Therefore, species can change as the population evolves.

3.3 Evolution Within Species

After species formation, genetic operations are applied independently within each species. Species with fewer than three individuals are skipped. Local evolution prevents one global route pattern from replacing all alternatives and allows each species to improve its own search direction.

3.3.1 Species-Level Structural Information Extraction

Before applying the genetic operators, GSA extracts structural information independently within each species. This design follows the genetic-engineering

principle introduced in GEA ^[4]. For a species of size N_s , the best 40% of individuals are selected as the local elite subset, with at least two elite individuals retained.

The frequency of an edge e in the local elite subset is defined as:

$$F(e) = \sum_{x \in P_{elite}} I(e \in E(x)). \quad (5)$$

where P_{elite} denotes the local elite subset, and $E(x)$ denotes the chromosome-edge set of individual x . The indicator function $I(e \in E(x))$ is equal to 1 if edge e appears in the chromosome-edge set of individual x , and 0 otherwise. Therefore, $F(e)$ represents the number of local elite individuals containing edge e .

The dominant chromosome is the elite individual with the highest total edge-frequency score. An informative-position mask is then constructed from the dominant chromosome. A customer position is marked as informative only if both adjacent edges appear in at least 55% of the local elite individuals.

This edge-frequency representation serves a role similar to the pattern matrix in the original GEA, but it is more suitable for routing problems because it captures customer adjacency relationships.

3.3.2 Standard Genetic Operators

Order crossover and swap mutation are first applied within each species. Order crossover preserves a segment from one parent and fills the remaining positions according to the order of another parent. Swap mutation exchanges randomly selected customer positions.

For each operator, the number of generated offspring is 25% of the species size,

rounded down, with at least one offspring generated.

3.3.3 Dominant-Chromosome Crossover

The first genetic-engineering operator corresponds to Scenario 1 in GEA^[4]. The locally extracted dominant chromosome is crossed with another individual from the same species using order crossover. The number of generated offspring is 50% of the species size, rounded down, with at least one offspring generated.

Since dominant chromosomes are extracted locally, different species can preserve different representative route patterns.

3.3.4 Directed Mutation

The second genetic-engineering operator corresponds to Scenario 2 in GEA^[4]. A parent is selected within the species using roulette-wheel selection. Directed mutation modifies only non-informative positions. It performs up to 30 2-opt-style segment-reversal attempts and up to 20 Or-opt-style relocation attempts. Each relocation moves a segment of one to three customers. A candidate move is accepted only if it improves the fitness.

The number of directed-mutation offspring is 30% of the species size, rounded down, with at least one offspring generated.

3.3.5 Gene Injection

The third genetic-engineering operator adapts the gene-injection concept of GEA^[4] to routing problems. A donor is selected from the local elite subset, and a target is selected from the weaker half of the species. A donor edge is eligible for injection if it is absent from the target and appears in at least 35% of the local elite individuals.

Edges connected to the depot are excluded from injection. For an eligible edge

(A, B) , customer (B) is removed from its current position and reinserted immediately before or after customer (A) . At most five edges are injected into one target.

The number of edge-injection offspring is 20% of the species size, rounded down, with at least one offspring generated.

3.3.6 Offspring Replacement

Every offspring is inserted through a novelty-aware replacement strategy. The child is accepted only if its average chromosome-edge similarity to the individuals in the current species is below 0.92. If this condition is satisfied, the child replaces the most structurally similar species member with worse fitness. This strategy avoids excessive duplication while allowing improved solutions to enter the species.

3.4 Inter-Species Interaction and Diversity Maintenance

3.4.1 Ring Migration

Migration is performed every 30 generations. Species are arranged in a ring topology. The best individual from each species is copied toward both neighboring species and replaces the worst target individual only if it has better fitness. This bidirectional migration shares useful information without fully mixing the population.

3.4.2 Species-Level Diversity Rescue

The diversity of a species S is measured using the average pairwise Jaccard distance:

$$Div(S) = \frac{2}{|S|(|S| - 1)} \sum_{i < j} D_{edge}(i, j). \quad (6)$$

If $Div(S)$ is below 0.04, the species is considered to have collapsed. The worst

30% of its individuals are replaced with random permutations. This mechanism reactivates species that have become overly homogeneous.

3.4.3 Global Diversity Injection and Burst

Two additional mechanisms maintain global diversity. At every generation, random candidates are generated for the worst 3% of the global population and inserted only if they improve the corresponding individuals. Every 80 generations, a stronger diversity burst replaces the worst 25% of the population with random permutations and forces reclustering. Global elitism ensures that the best solution found so far is preserved.

3.5 Overall Framework

The complete GSA procedure is summarized in Algorithm 1. The algorithm first initializes and evaluates a random population, and records the global best solution. During evolution, the population is periodically reclustered into species using the Wishart-style clustering procedure. When the diversity-burst interval is reached, the worst part of the population is replaced by random individuals and reclustering is forced. Within each species, diversity rescue is applied when species diversity becomes too low. Then, local structural information is extracted and used to generate offspring through standard and genetic-engineering operators. Offspring are inserted by a novelty-aware replacement strategy. At fixed intervals, bidirectional ring migration allows limited information exchange between species. Global random injection and elitist preservation are also applied to maintain diversity and keep the best solution found so far. The algorithm stops after the maximum number of generations and returns

the best solution.

Algorithm 1. Genetic Speciation Algorithm

```

Input: problem instance; population size  $N$ ; maximum generations  $G$ 
Output: best solution found

Initialize and evaluate a random population
Record the global best solution
for generation  $g = 0, 1, \dots, G - 1$  do
    if the diversity-burst interval is reached then
        Replace the worst 25% of the population with random individuals
        Force reclustering
    end if
    if the reclustering interval is reached then
        Form species using the Wishart-style clustering procedure
    end if
    for each species  $S$  do
        Apply diversity rescue if  $Div(S) < 0.04$ 
        Extract local structural information
        Generate offspring using standard and genetic-engineering operators
        Insert offspring using novelty-aware replacement
    end for
    if the migration interval is reached then
        Perform bidirectional ring migration
    end if
    Apply global random injection
    Preserve the global best solution
end for
return the best solution

```

And the main parameter settings used in the experiments are summarized in Table 1. The population size and maximum number of generations define the overall computational budget. The parameters related to species formation control the balance between structural similarity and fitness information, as well as the number and update frequency of species. The remaining parameters determine how local structural information is extracted and when diversity recovery is triggered. These settings were applied consistently across all benchmark instances to ensure a fair comparison.

Table 1. Main parameter settings of GSA.

Parameter	Value
Population size N	200
Maximum generations G	500
Hybrid-distance weight α	0.2
Maximum number of species	5
Minimum species size	3
Reclustering interval	20 generations
Migration interval	30 generations
Diversity-burst interval	80 generations
Local elite ratio	0.4
Informative-edge threshold	0.55
Novelty-similarity threshold	0.92
Species-diversity collapse threshold	0.04

4 Experimental Results

This section reports the computational results of the proposed Genetic Speciation Algorithm (GSA). The experiments are designed to examine whether dynamic speciation improves solution quality and robustness in combinatorial optimization. The proposed method is evaluated on standard capacitated vehicle routing problem (CVRP) benchmark instances and compared with two baseline algorithms.

4.1 Experimental Setup

Three algorithms are included in the comparison. The first is a standard genetic algorithm (GA), which uses conventional crossover and mutation operators. The second is a non-speciated genetic engineering algorithm (GEA), which uses enhanced genetic operators but evolves the population as one mixed group. The third is the proposed GSA, which introduces dynamic species formation into the evolutionary process.

The experiments use three instances from the Augerat A-set CVRP benchmark: A-n32-k5, A-n60-k9, and A-n80-k10 [6]. These instances contain 31, 59, and 79 customers, respectively. The best known solutions are used as reference values for evaluating solution quality. For all algorithms, the population size is 200, the maximum number of generations is 500, and each algorithm is run five independent times on each instance.

The performance is evaluated using the best objective value, mean objective value, standard deviation, gap to the best known solution, and average runtime. Lower objective values and lower gaps indicate better performance.

4.2 Overall Performance

Table 2 summarizes the main numerical results. GSA obtains the best mean objective value on all three benchmark instances. It also gives the smallest standard deviation, which indicates more stable performance across independent runs.

Table 2. Experimental results summary on the Augerat CVRP instances.

Instance	Algorithm	Best	Mean	Std. Dev.	Gap to BKS	Avg. Time
A-n32-k5	GA	944.6	1027.9	41.79	31.1%	4.5s
A-n32-k5	GEA	857.6	907.1	28.99	15.7%	12.1s
A-n32-k5	GSA	803.3	823.6	17.27	5.1%	55.5s
A-n60-k9	GA	2110.1	2184.1	48.12	61.3%	6.8s
A-n60-k9	GEA	1517.2	1613.0	50.63	19.1%	17.5s
A-n60-k9	GSA	1463.7	1499.0	30.72	10.7%	90.9s
A-n80-k10	GA	3104.7	3161.3	36.98	79.3%	8.0s
A-n80-k10	GEA	2078.1	2190.9	79.58	24.3%	20.7s
A-n80-k10	GSA	1978.1	2004.2	25.30	13.7%	132.4s

Figure 1 show the comparison with the non-speciated GEA, GSA reduces the mean gap to the best known solution from 15.7 percent, 19.1 percent, and 24.3 percent to 5.1 percent, 10.7 percent, and 13.7 percent, respectively. These results suggest that the improvement comes not only from stronger genetic operators, but also from the population organization introduced by speciation.

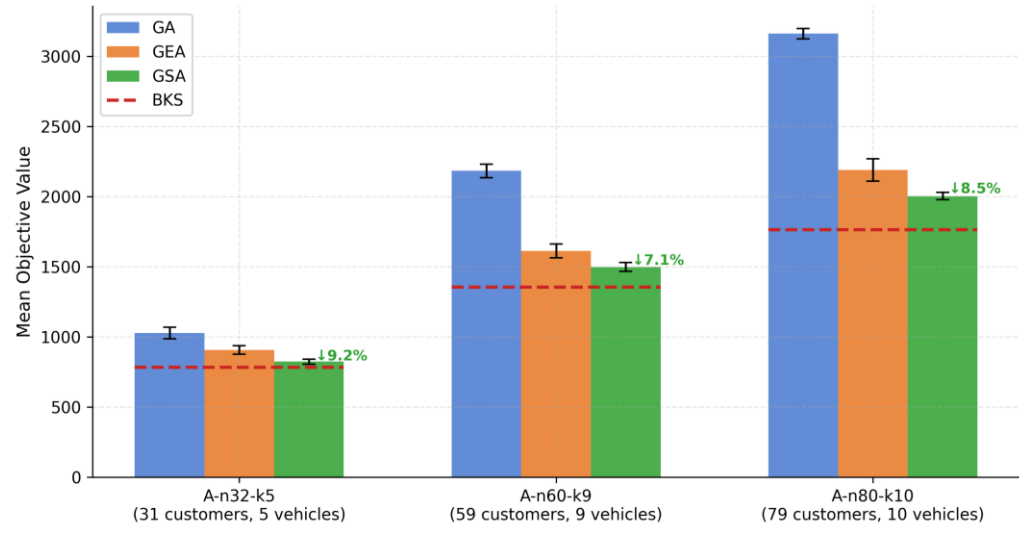


Figure 1. Comparison of mean objective values with standard deviations.

4.3 Convergence Behavior

Figure 2 show how the algorithms improve during the search process. GA improves slowly and often remains far from the best known solution. GEA converges faster because it uses more informed search operators. GSA further improves the convergence behavior by preserving multiple species, so different route structures can continue to develop instead of being eliminated early.

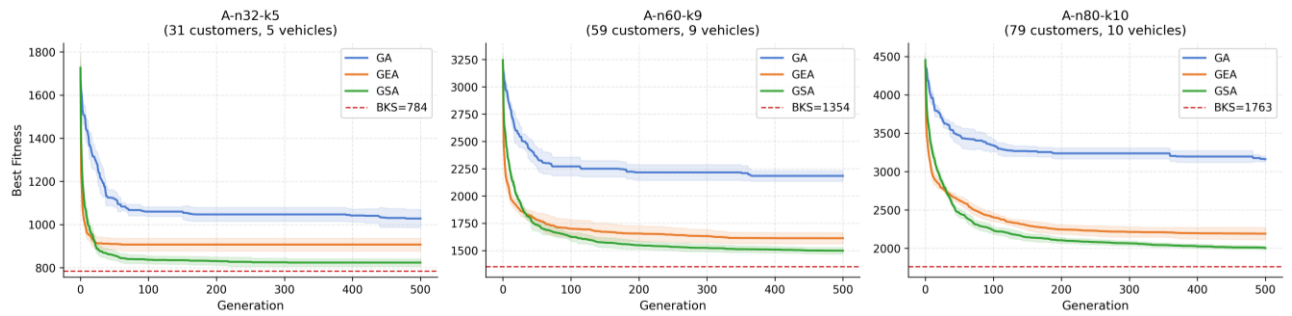


Figure 2. Mean convergence curves of GA, GEA, and GSA on all benchmark instances.

4.4 Robustness and Scalability

Figure 3 show the distribution of final objective values across independent runs. GSA has a tighter distribution than the comparison algorithms, especially on the larger instances. This means that the proposed speciation mechanism improves not only the best observed result, but also the reliability of the algorithm.

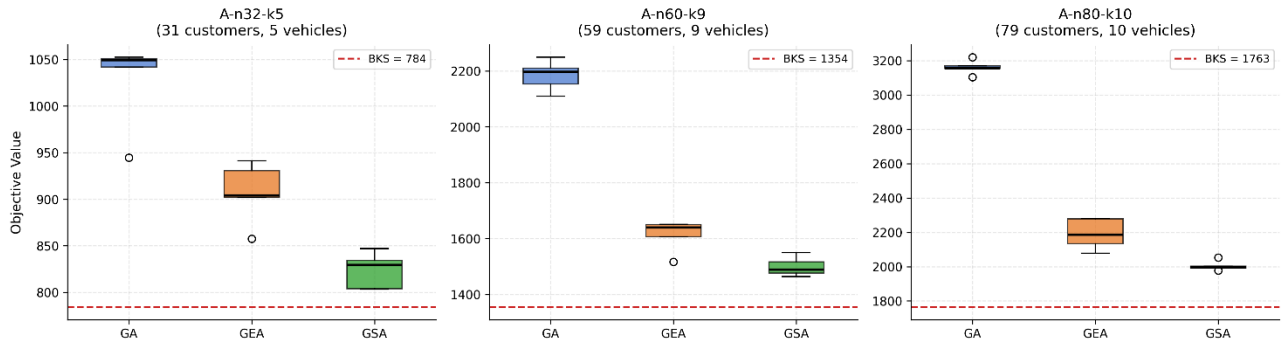


Figure 3. Distribution of final solutions across independent runs.

Figure 4 compare algorithm behavior as the number of customers increases. The gap to the best known solution grows rapidly for GA, while GEA and GSA remain more competitive. GSA consistently achieves the smallest gap, showing that preserving multiple structural search directions becomes increasingly useful as the search space grows.

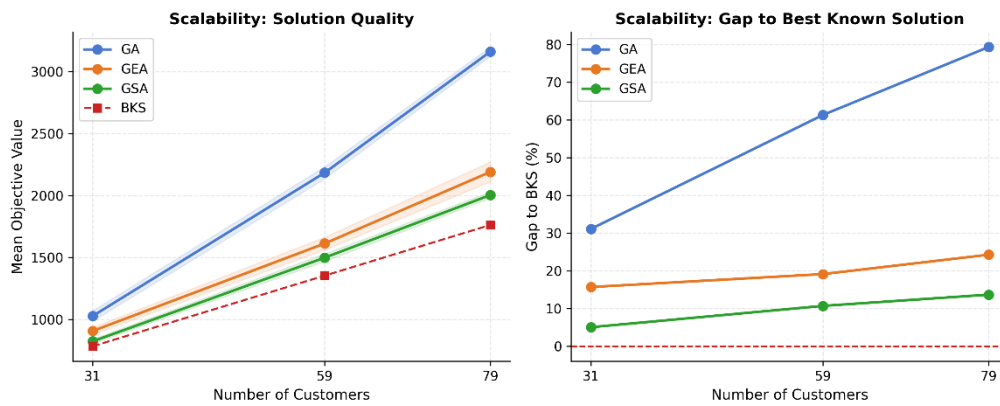


Figure 4. Scalability and gap to the best known solutions.

4.5 Species Dynamics

Figures 5 describe the behavior of the species during the GSA search process. The number of species remains within a controlled range, while intra-species diversity changes over time. This indicates that the species are not static labels; they are adjusted as the population evolves.

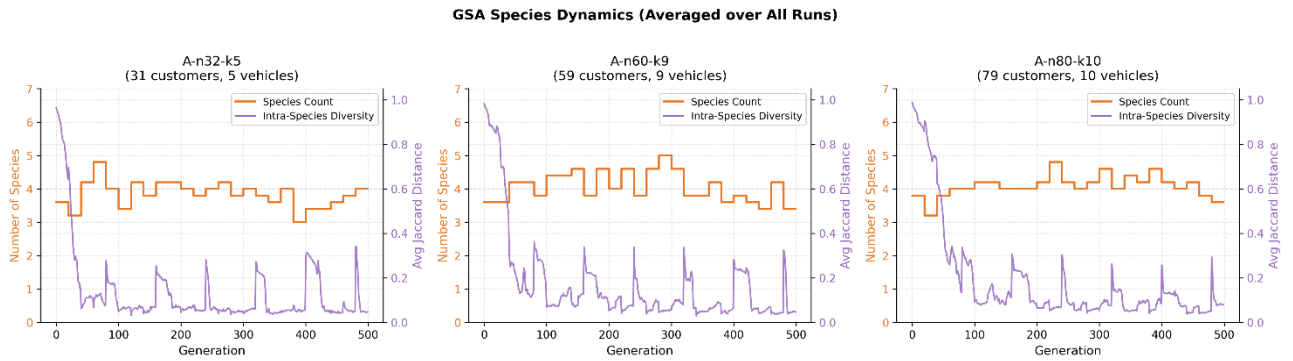


Figure 5. Species count and intra-species diversity during evolution.

Figure 6 show the species size evolution, different species occupy different portions of the population at different generations. This dynamic behavior is important because it allows the algorithm to keep several search niches active while still adapting to the current distribution of solutions.

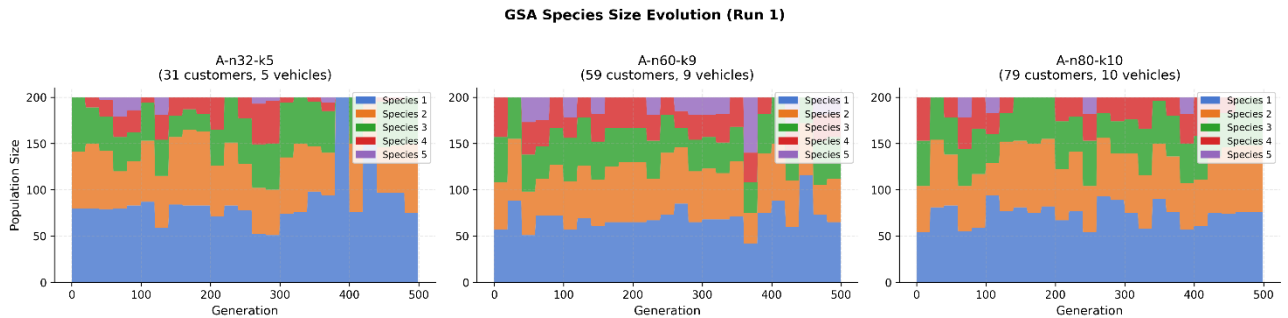


Figure 6. Species size evolution during the search process.

4.6 Discussion

Overall, the experimental results support the effectiveness of speciation in GSA. By dividing the population into structurally meaningful species, the algorithm avoids relying on one dominant search direction. Each species can refine its own route pattern, while limited interaction between species allows useful structures to spread. This improves the balance between exploration and exploitation.

The main cost of GSA is runtime. Species formation, diversity monitoring, and inter-species interaction make GSA slower than GA and GEA. However, the additional cost leads to better objective values, smaller gaps, and more stable results. For combinatorial optimization problems where solution quality is more important than very short runtime, this trade-off is reasonable.

5 Conclusion and Future Work

This paper proposed a Genetic Speciation Algorithm (GSA) for combinatorial optimization problems. The main idea of GSA is to introduce speciation into the genetic search process. Instead of evolving all individuals in a single mixed population, GSA dynamically divides the population into several species according to solution-structure similarity and fitness information. Each species preserves a different search direction and evolves within its own search niche.

The proposed method was applied to the capacitated vehicle routing problem. In this problem, candidate solutions were represented as customer permutations, and species were mainly formed according to route-edge similarity. This design allowed the algorithm to group individuals based on meaningful routing structures rather than simple positional similarity. As a result, different route patterns could be maintained and improved during the search process.

Experimental results on standard CVRP benchmark instances showed that GSA outperformed both the standard genetic algorithm and the non-speciated genetic engineering algorithm. Compared with these methods, GSA achieved lower mean objective values, smaller gaps to the best known solutions, and lower standard deviations across independent runs. These results indicate that speciation can effectively preserve structural diversity, reduce premature convergence, and improve the robustness of genetic search.

The main limitation of GSA is its additional computational cost. Species formation, diversity monitoring, and inter-species interaction require more time than a

standard genetic algorithm. However, the experimental results suggest that this cost is reasonable when solution quality and stability are important.

Future work can be extended in several directions. First, GSA should be tested on larger CVRP instances and other combinatorial optimization problems, such as scheduling, assignment, and facility location. Second, the parameters related to species formation, migration frequency, and diversity recovery can be further studied. Third, adaptive strategies may be introduced so that the number of species and the clustering frequency can change automatically during the search process. Finally, since different species can evolve with partial independence, parallel implementation is a promising direction for improving the computational efficiency of GSA.

Reference

- [1] Du, D. Z., and Pardalos, P. M. Handbook of Combinatorial Optimization. Springer, 1998.
- [2] Holland, J. H. Adaptation in Natural and Artificial Systems. University of Michigan Press, 1975.
- [3] Katoch, S., Chauhan, S. S., and Kumar, V. A. A Review on Genetic Algorithm: Past, Present, and Future. Multimedia Tools and Applications, 80, 8091-8126, 2021.
- [4] Sohrabi, M., Fathollahi-Fard, A. M., and Gromov, V. A. Genetic Engineering Algorithm (GEA): An Efficient Metaheuristic Algorithm for Solving Combinatorial Optimization Problems. Automation and Remote Control, 85(3), 252-262, 2024.
- [5] Wishart, D. Mode Analysis: A Generalization of Nearest Neighbor Methods for Use in Clustering and Pattern Recognition. In Numerical Taxonomy, Academic Press, 1969.
- [6] Augerat, P., Belenguer, J. M., Benavent, E., Corberan, A., Naddef, D., and Rinaldi, G. Computational Results with a Branch-and-Cut Code for the Capacitated Vehicle Routing Problem. Research Report 949-M, Universite Joseph Fourier, Grenoble, 1995.